

UNIVERSIDADE BRAZ CUBAS

EDUARDO FELIPE BRAGA SILVA / RGM: 43733514]

EDUARDO GABRIEL S.C / RGM: 45740321

GABRIEL JOSÉ C. PEREIRA / RGM: 43809707

TICKFLOW

Mogi das Cruzes, SP.

2026

TICKFLOW

Proposta de Projeto RFC apresentada à
disciplina de Estrutura de Dados I, sob
orientação de Andrea Ono Sakai.

Mogi das Cruzes, SP.

2026

SUMÁRIO

1	NOME DO SISTEMA
2	PROBLEMA
2.1	Descrição do Problema
2.2	Quem Enfrenta Esse Problema?
2.3	Frequência e Impacto
2.4	Importância da Solução
3	SOLUÇÃO PROPOSTA
3.1	Visão Geral do Sistema
3.2	Fluxo Principal da Aplicação
3.3	Módulos Principais
3.4	Interação do Usuário
4	USO DAS ESTRUTURAS DE DADOS
4.1	Lista
4.2	Fila
4.3	Pilha
5	COMPLEXIDADE COMPUTACIONAL
5.1	Operações Custosas Identificadas
5.2	Análise de Escalabilidade
5.3	Dashboard de Desempenho
6	CONCLUSÃO
6.1	Resumo da Proposta
6.2	Principais Benefícios
6.3	Viabilidade Técnica
7	CHECKLIST DE AUTOAVALIAÇÃO

1 NOME DO SISTEMA

TICKFLOW

2 PROBLEMA

2.1 Descrição do Problema

A venda de ingressos para shows e eventos de grande porte enfrenta um problema crítico: quando muitos usuários tentam comprar ao mesmo tempo, sistemas sem controle adequado falham em garantir justiça e consistência. Isso resulta em ingressos vendidos em duplicidade, sobrecarga dos servidores, filas desorganizadas e frustração dos compradores. Sem uma estrutura bem definida para gerenciar o fluxo de compras, o sistema se torna impraticável em cenários de alta demanda.

2.2 Quem Enfrenta Esse Problema?

O problema afeta dois grupos principais. O primeiro são os usuários finais, pessoas comuns que tentam comprar ingressos para shows, festivais e eventos culturais que enfrentam lentidão, erros e injustiça no processo de compra. O segundo são os organizadores de eventos, que precisam garantir que a venda ocorra de forma ordenada, sem duplicidade de ingressos e com controle real do número de entradas disponíveis

2.3 Frequência e Impacto

O problema ocorre sempre que um evento de alta demanda abre vendas, o que pode acontecer diversas vezes por mês em plataformas de grande porte. O impacto é significativo: sem controle de fila, usuários mais lentos perdem ingressos para bots ou para concorrentes que exploram falhas do sistema. Além disso, a ausência de um mecanismo de reserva temporária pode resultar em dois usuários comprando o mesmo ingresso simultaneamente, gerando conflitos de dados e prejuízos financeiros e de reputação para o organizador.

2.4 Importância da Solução

Resolver esse problema é essencial para garantir uma experiência justa e confiável para todos os usuários.

Um sistema bem estruturado elimina a possibilidade de ingressos duplicados, organiza o atendimento por ordem de chegada e permite que os organizadores monitorem em tempo real o status das vendas. Além disso, a utilização de estruturas de dados adequadas como fila, lista e pilha torna o sistema escalável, capaz de suportar grandes volumes sem perda de desempenho.

3 SOLUÇÃO PROPOSTA

3.1 Visão Geral do Sistema

O Sistema de Ingressos é uma aplicação web desenvolvida em Python com o objetivo de gerenciar a venda de entradas para shows e eventos de forma organizada, justa e eficiente. O sistema permite que usuários visualizem o catálogo de eventos disponíveis, entrem em uma fila de compra respeitando a ordem de chegada, adquiram ingressos e gerenciem suas ações com a possibilidade de desfazê-las. Internamente, o sistema utiliza três estruturas de dados fundamentais: Lista, Fila e Pilha implementadas manualmente, além de algoritmos de ordenação para organizar os eventos por diferentes critérios.

O sistema de ingressos tem como objetivo gerenciar a venda de entradas para shows e eventos de forma organizada e direta. Ele permite que usuários visualizem os eventos disponíveis, entrem em uma fila de compra, adquiram ingressos e gerenciem suas ações dentro do sistema.

3.2 Fluxo Principal da Aplicação

- 1. O usuário abre o sistema e visualiza o catálogo de eventos disponíveis, exibido em formato de lista ordenada.*
- 2. O usuário aplica filtros para encontrar o evento de interesse, esses são: preço, data, popularidade ou pelo nome do evento/show.*
- 3. Ao selecionar o evento, o usuário entra numa fila de compra respeitando a ordem de chegada (FIFO).*
- 4. O sistema atende os usuários por ordem de chegada, liberando a compra quando sua vez chega.*
- 5. O usuário escolhe a quantidade de ingressos, assentos (se houver) e confirma a compra.*
- 6. Caso o usuário cometa algum erro, ele pode desfazer a ação com o método pilha (LIFO).*
- 7. Após a compra, o sistema registra o histórico do usuário (com pilha).*

3.3 Módulos Principais

Módulo 1: Catálogo de Eventos. (Lista)

Armazena todos os eventos disponíveis, cada evento contém dados como nome, data, preço, assentos e quantidade de ingressos. Também permite ordenação.

Módulo 2: Sistema de Fila de Compra (FIFO)

Limita a fila de usuário para organizar as compras de ingressos, evitando conflitos de sobrecarga ou ingressos duplos.

Módulo 3: Controle de Ações e Histórico (Pilha)

Armazena as ações do usuário e permite desfazer as mesmas, também mantém um histórico de compras e interações.

Módulo 4: Sistema Principal de Ordenação.

Implementação dos algoritmos. Como o Bubble Sort e Quick Sort, a fim de organizar o desempenho das compras de ingressos.

3.4 Interação do Usuário

Visualizar a lista de eventos disponíveis, Filtrar e ordenar eventos por diferentes critérios, Selecionar um evento de interesse, Entrar na fila de compra, Comprar ingressos quando chegar sua vez, Desfazer ações (como seleção incorreta), Visualizar seu histórico de compras e o status do ingresso.

4 USO DAS ESTRUTURAS DE DADOS

4.1 Lista

O que é armazenado na lista?

A lista armazena os objetos do tipo Evento, cada um contendo: nome do evento, artista, data e horário, local, preço do ingresso, número total de ingressos disponíveis. É a estrutura central do catálogo de eventos do sistema.

Por que usar lista?

A lista é a estrutura ideal para o catálogo de eventos pois precisa manter ordem de inserção, permitir acesso por posição (para exibir eventos em ordem), percorrer

todos os elementos (para aplicar filtros e ordenação) e inserir ou remover eventos em posições específicas conforme eventos são cadastrados ou encerrados.

- *Precisa manter ordem de inserção (x)*
- *Precisa acessar elementos por posição (x)*
- *Precisa percorrer todos os elementos (x)*
- *Precisa inserir/remover em posições específicas (x)*
- *Outro motivo: facilidade para aplicar algoritmos de ordenação (preço, data, popularidade.)*

Operações Principais

Inserção: ao final da lista. Ocorre quando o organizador cadastrar um novo evento no sistema.

Remoção: por posição ou por ID do evento. Ocorre quando um evento é cancelado, encerrado ou esgotado.

Busca: por nome do evento, artista, data ou faixa de preço. Ocorre quando o usuário aplica filtros na tela de catálogo para encontrar um evento de interesse.

Travessia: percorrer todos os elementos da lista para aplicar algoritmos de ordenação (Bubble Sort ou Quick Sort) e exibir os eventos organizados por preço, data ou popularidade.

Exemplo Concreto

O usuário acessa a tela inicial e o sistema percorre a lista de eventos para exibir todos os shows disponíveis. Em seguida, o usuário aplica o filtro “ordenar por preço” e o algoritmo Quick Sort é executado sobre a lista, reordenando os nós. O usuário visualiza o resultado e seleciona o evento “Show do Metallica R\$250,00”, que ocupa a posição 3 da lista após a ordenação.

4.2 Fila

O que é enfileirado?

São enfileirados objetos do tipo Usuário aguardando compra, contendo: ID do usuário, nome, o evento de interesse e o momento de entrada na fila (timestamp). Cada posição na fila representa um usuário aguardando sua vez para realizar a compra do ingresso.

Por que usar fila (FIFO)?

O processo deve ser FIFO pois garante que os usuários sejam atendidos por ordem de chegada, evitando injustiças ou conflitos na compra de ingressos.

Sendo assim, o primeiro usuário a entrar na fila deve ser o primeiro a ser atendido. Qualquer outra ordem seria injusta e abriria brechas para que usuários mais recentes pulassem a vez de quem chegou antes.

Operações Principais

Queue (enfileirar): o usuário é enfileirado ao clicar em “Comprar Ingresso” em um evento. O sistema registra o timestamp e insere o usuário no final da fila.

Dequeue (desenfileirar): o usuário do início da fila é removido e liberado para a tela de compra. Ocorre quando o sistema termina de atender o usuário atual e chama o próximo.

Peek (espiar): utilizado para exibir ao usuário sua posição atual na fila e o tempo estimado de espera, sem removê-lo. Também usado pelo sistema para verificar quem será atendido a seguir.

Por que FIFO é essencial neste caso?

Sem o FIFO, o sistema poderia atender usuários de forma aleatória ou por prioridade incorreta, gerando injustiça e conflitos. Imagine que dois usuários entram na fila ao mesmo tempo para o mesmo show com apenas um ingresso restante: sem FIFO, o sistema não teria critério claro para decidir quem compra. Com FIFO, o problema é resolvido deterministicamente, quem chegou primeiro, leva.

Exemplo Concreto

As vendas do show “Rock in Rio” abrem às 10h. Três usuários clicam em “Comprar” quase simultaneamente: Ana (10:00:01), Bruno (10:00:02) e Carlos (10:00:03). O sistema os enfileira nessa ordem. Ana é atendida primeiro (dequeue), realiza a compra e Carlos obtém posição 2 na fila com previsão de espera exibida na tela via peek.

4.3 Pilha

O que é empilhado?

São empilhados objetos do tipo Ação, cada um representando uma operação realizada pelo usuário: seleção de evento, escolha de quantidade de ingressos, aplicação de filtro ou confirmação de compra. Cada ação registra o tipo da operação e o estado anterior do sistema antes dela.

Por que usar pilha (LIFO)?

O processo deve ser LIFO pois ao desfazer ações, a ordem natural é reversa: a última coisa feita é a primeira a ser desfeita. Isso reflete exatamente o comportamento

esperado pelo usuário, assim como o Ctrl+Z em qualquer editor. Não faria sentido desfazer uma ação antiga antes de desfazer a mais recente.

Operações Principais

Push (empilhar): cada ação do usuário é empilhada imediatamente após ser executada.

Ocorre ao selecionar um evento, alterar a quantidade de ingressos ou aplicar um filtro.

Pop (desempilhar): a ação do topo é removida e o sistema retorna ao estado anterior.

Ocorre quando o usuário clica em “Desfazer” ou pressiona o botão de voltar dentro do fluxo de compra.

Peek (espiar topo): utilizado para exibir ao usuário qual será a próxima ação desfeita caso ele clique em “Desfazer”, sem ainda executá-la. Também usado para exibir o último item do histórico de compras.

Por que LIFO é essencial neste caso?

Se o sistema usasse uma fila (FIFO) para o histórico de ações, ao desfazer, seria revertida a primeira ação feita, o que é contra-intuitivo e potencialmente destrutivo. O usuário que selecionou um evento errado quer desfazer apenas a última ação, não reiniciar tudo desde o começo. A pilha garante que o desfazer funcione como o usuário naturalmente espera: do mais recente para o mais antigo.

Exemplo Concreto

O usuário acessa o sistema e realiza três ações em sequência: (1) seleciona o evento “Festival Lollapalooza”, (2) escolhe 3 ingressos, (3) altera para 2 ingressos. A pilha armazena essas ações nessa ordem. Ao clicar em “Desfazer”, o sistema faz pop da ação (3) e volta para 3 ingressos. Clicando novamente, faz pop de (2) e desseleciona a quantidade. A pilha garante que cada desfazer reverte exatamente a última decisão tomada.

5 COMPLEXIDADE COMPUTACIONAL

5.1 Operações Custosas Identificadas

Operação 1: Ordenação dos eventos (por preço, data ou popularidade)

Operação 2: Busca de eventos na lista (por nome ou filtros)

Operação 3: Processamento da fila de usuários (principalmente com muitos acessos simultâneos)

a) *Ordena os Eventos (por um filtro que inclui: preço, data ou popularidade.)*

- b) *Busca De Eventos (pelos mesmos filtros.)*
- c) *Processamento da fila de usuários.*

5.2 Análise de Escalabilidade

Operação	Estrutura	Complexidade	Justificativa	Impacto (100k)
<i>Ordenação de Eventos.</i>	<i>Lista</i>	<i>$O(n^2)$ (Bubble) $O(n \log n)$ (Quick)</i>	<i>BubbleSort compara todos os elementos várias vezes, enquanto Quicksort divide o problema em partes menores</i>	<i>Bubble: muito lento (~segundos). Quick: rápido (~milissegundos)</i>
<i>Busca de Eventos</i>	<i>Lista</i>	<i>$O(n)$</i>	<i>Percorre a linha inteira no pior caso para encontrar o evento.</i>	<i>Causa demora perceptível.</i>
<i>Gestão da Fila</i>	<i>Fila (FIFO)</i>	<i>$O(1)$</i>	<i>Inserção e remoção acontecem diretamente nas extremidades.</i>	<i>Rápido, mas pode gerar grande fila.</i>

5.3 Dashboard de Desempenho

Tamanho da Fila (quantidade de usuários aguardando), Tempo de Médio de Espera (quanto tempo usuário espera para operar) e Organização de Atendimentos (quantos usuários conseguem operar por segundo).

Métricas de Fila:

- *Tamanho da fila: quantidade de usuários aguardando atendimento em tempo real.:*
- *Tempo médio de espera: quanto tempo cada usuário aguarda na fila até ser atendido.:*
- *Vazão de atendimento: quantos usuários são processados por segundo.:*

Métricas de Lista:

- *Tempo de ordenação da lista: tempo gasto para ordenar os eventos por preço, data ou popularidade usando Bubble Sort ou Quick Sort.*
- *Tempo de busca na lista: tempo necessário para localizar um evento por nome ou filtro, percorrendo a lista linearmente $O(n)$.*
- *Total de eventos cadastrados: número atual de eventos na lista, indicando o volume de dados sobre o qual os algoritmos de ordenação operam.*

Métricas de Pilha:

- *Quantidade de ações armazenadas: número de ações no histórico.*
- *Tempo de desfazer ação: tempo para executar um “desfazer”.*
- *Número de operações de desfazer: quantas vezes usuários utilizam essa função.*

Alertas de Performance:

Alerta 1: fila muito grande (ex: mais de 10.000 usuários) → indica sobrecarga

Alerta 2: tempo de ordenação alto → indica algoritmo ineficiente (ex: uso de Bubble Sort em muitos dados)

Indicador positivo: tempo médio de atendimento baixo → sistema fluindo bem

- Alerta 1: Fila Com Sobrecarga. (Mais de 100k por exemplo).
- Alerta 2: Alto Tempo de Ordenação. (algoritmo ineficiente.)
- Alerta 3: Positivo. (Tempo médio de operações baixo).

6 CONCLUSÃO

6.1 Resumo da Proposta

O TICKFLOW é uma aplicação web em Python para gerenciar a venda de ingressos para shows de forma justa e eficiente. Utiliza Lista, Fila e Pilha implementadas manualmente para exibir eventos, controlar a ordem de compra e permitir desfazer ações. A ordenação é feita com Bubble Sort e Quick Sort, comparando algoritmos $O(n^2)$ e $O(n \log n)$ em um dashboard de telemetria em tempo real.

6.2 Principais Benefícios

Os três principais benefícios do sistema são listados a seguir.

- 1. Justeza e consistência na venda: a Fila FIFO garante que nenhum usuário seja atendido fora de ordem, eliminando conflitos e a possibilidade de ingressos duplos.*
- 2. Experiência de usuário aprimorada: a Pilha LIFO permite desfazer ações de forma intuitiva, reduzindo erros de compra e aumentando a confiança do usuário no sistema.*
- 3. Desempenho observável e escalável: o dashboard de telemetria permite identificar gargalos em tempo real, e o uso do Quick Sort $O(n \log n)$ garante ordenação eficiente mesmo com grandes volumes de eventos cadastrados.*

6.3 Viabilidade Técnica

Viabilidade: Média. As estruturas de dados (Lista, Fila e Pilha) são conceitos já estudados na disciplina e sua implementação manual em Python é viável no prazo do projeto. O maior desafio está na construção da interface web e do dashboard de desempenho com telemetria em tempo real, o que exige integração entre o back-end Python e o front-end. A não utilização do uso de funções prontas de ordenação (como `sort()`) demanda atenção redobrada na implementação dos algoritmos. No geral, o projeto é tecnicamente viável para a equipe dentro do prazo estabelecido.

7 CHECKLIST DE AUTOAVALIAÇÃO

Antes de entregar, verifique se você:

7.1 Clareza e Viabilidade

- O problema está claramente definido? (x)
- A solução é viável e faz sentido? (x)
- O fluxo do sistema está claro? (x)

7.2 Uso das Estruturas de Dados

- Lista é usada de forma justificada (não forçada)? (x)
- Fila representa um processo FIFO real? (x)
- Pilha representa um comportamento LIFO real? (x)
- Todas as operações estão bem explicadas? (x)

7.3 Análise de Complexidade

- Operações custosas foram identificadas? (x)
- Complexidade de cada operação está correta? (x)
- Análise de escalabilidade é realista? (x)
- Dashboard de desempenho é relevante? (x)

7.4 Qualidade da Escrita

- Texto claro e objetivo? (x)
- Sem erros gramaticais? (x)
- Exemplos concretos fornecidos? (x)
- Todas as seções foram preenchidas? (x)